When dividing work across threads by task type, you don't have to limit yourself to completely isolated cases. If multiple sets of input data require the same *sequence* of operations to be applied, you can divide the work so each thread performs one stage from the overall sequence.

Dividing a Sequence of Tasks Between Threads

If your task consists of applying the same sequence of operations to many independent data items, you can use a *pipeline* to exploit the available concurrency of your system. This is by analogy to a physical pipeline: data flows in at one end through a series of operations (pipes) and out at the other end.

To divide the work this way, you create a separate thread for each stage in the pipeline—one thread for each of the operations in the sequence. When the operation is completed, the data element is put on a queue to be picked up by the next thread. This allows the thread performing the first operation in the sequence to start on the next data element while the second thread in the pipeline is working on the first element.

This is an alternative to just dividing the data between threads, as described in section 8.1.1, and is appropriate in circumstances where the input data itself isn't all known when the operation is started. For example, the data might be coming in over a network, or the first operation in the sequence might be to scan a filesystem in order to identify files to process.

Pipelines are also good where each operation in the sequence is time consuming; by dividing the tasks between threads rather than the data, you change the performance profile. Suppose you have 20 data items to process, on four cores, and each data item requires four steps, which take 3 seconds each. If you divide the data between four threads, then each thread has 5 items to process. Assuming there's no other processing that might affect the timings, after 12 seconds you'll have 4 items processed, after 24 seconds 8 items processed, and so forth. All 20 items will be done after 1 minute. With a pipeline, things work differently. Your four steps can be assigned one to each processing core. Now the first item has to be processed by each core, so it still takes the full 12 seconds. Indeed, after 12 seconds you only